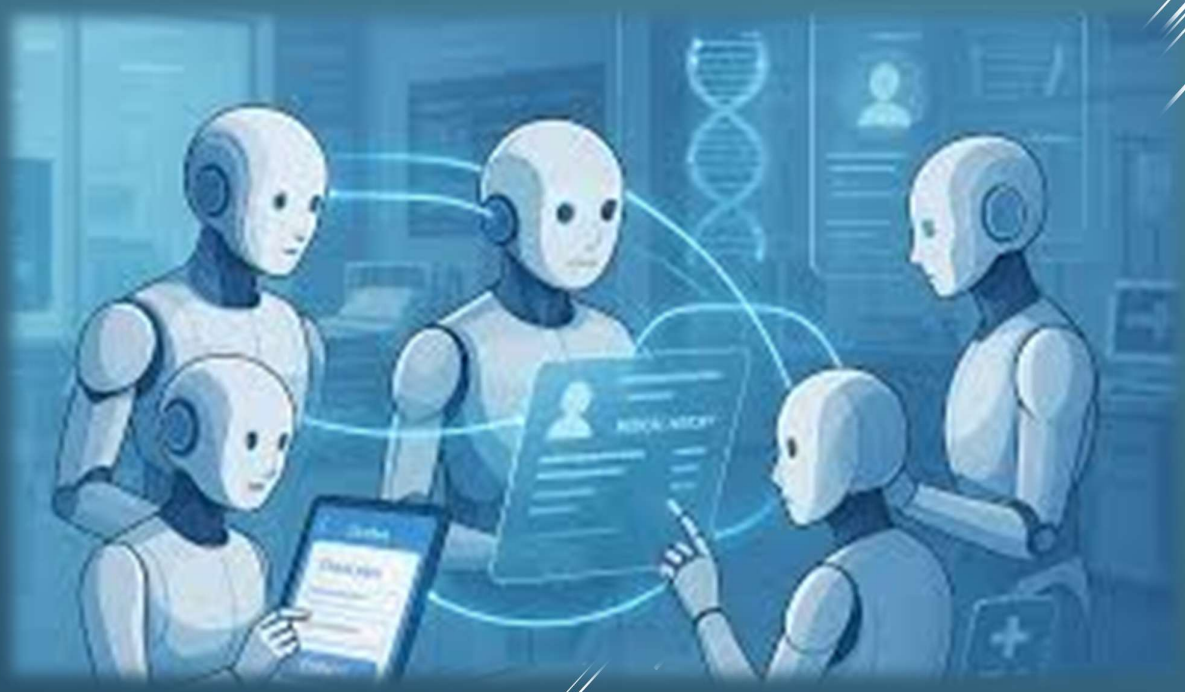


Building AI That Works Together

Lessons from Developing a Multi-Agent Collaboration System

Building AI That Works Together

Lessons from Developing a Multi-Agent Collaboration System

Craig Yappert | November 2025

The Winding Road to the Problem

I didn't start by building a multi-agent system. I started where most people start in 2025: with a chat interface, asking an AI to "build me an app."

That worked until it didn't. The chat window was great for questions and simple tasks but fell apart for anything requiring sustained context or iterative development. So, I moved to IDE-integrated tools—Windsurf, Cursor, Kiro—hoping tighter integration with my development environment would solve the coordination problem.

It helped, but introduced new friction. Each tool had opinions about workflow. Context is still fragmented between sessions. And I found myself managing the AI as much as using it.

Eventually, I settled on Claude Code within VS Code, which gave me the right balance of power and flexibility. But even then, a core frustration remained: different AI models excel at different things: Claude for reasoning, Copilot for code completion, specialized models for specific domains, and they couldn't talk to each other. I was the integration layer, copying context between tools, manually coordinating outputs.

It's remarkable how much the basic chat interface still dominates AI interaction in 2025. Organizations are running production workflows through the same paradigm I started with and hitting the same walls.

That frustration became the challenge: to build a system where multiple AI agents can collaborate in VS Code, each sharing their expertise while keeping shared context. What started as a focused weekend project turned into a four-month journey that taught me more about practical AI development than any documentation ever could.

What I Built

The Multi Agent Chat Extension began as a fork of André Pimenta's Claude Code Chat. It evolved through 150+ commits into a different tool: a VS Code extension that allows seven specialized AI agents (Architect, Coder, Executor, Reviewer, Documenter, Coordinator, and Team) to collaborate via natural “@mentions”. It supports over 28 models from six providers, including zero-cost GitHub Copilot integration.

But the real story isn’t in the feature list. It's what I learned by getting it wrong, very wrong, first.

Lesson 1: Optimizing for the Wrong Thing

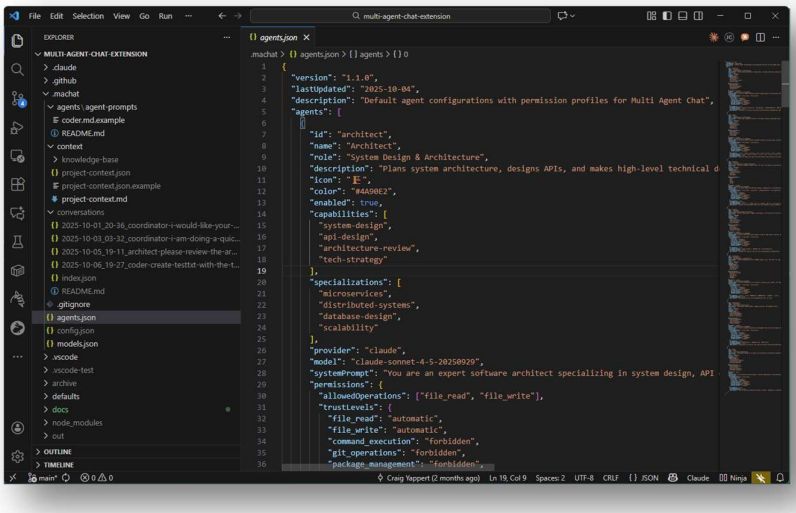
My first architectural instinct was performance. I built an entire MCP (Model Context Protocol) server with WebSocket connections, caching layers, and Express endpoints. It was technically impressive, about 500KB of infrastructure code designed to shave milliseconds off response times.

Then I used it. And realized something uncomfortable: when AI eliminates days of human work, whether it takes 30 seconds or 3 minutes is irrelevant. I was solving the wrong problem.

> *The entire MCP infrastructure was deleted in v1.11.0. The codebase got simpler. The debugging got easier. Nothing of value was lost.*

The takeaway: Speed optimization in AI systems is often premature. Quality, reliability, and transparency matter more than raw performance. Being willing to delete working code that solves the wrong problem is more complicated than it sounds, but essential.

Lesson 2: Trust Is Earned, Not Assumed



Earlier in my AI development journey (before this project), I learned the security lesson the hard way. I was new to AI-assisted development, trying to learn software engineering patterns while simultaneously trusting AI to execute operations. I gave it access. It deleted a database.

The AI wasn't malicious. It misunderstood the context, or I miscommunicated the intent—honestly, it doesn't matter which. The result was the same: real damage from misplaced trust. I was trying to coordinate something I didn't fully understand, and I assumed the AI's competence extended to judgment about consequences.

By the time I built the Multi-Agent Chat Extension, I had scars. So security was foundational: encrypted API key storage, workspace boundary enforcement, operation logging, and permission systems by agent type. When a path traversal vulnerability was discovered in October, it was caught because security auditing was part of the development process, not an afterthought.

The takeaway: AI systems that can execute code or modify files need graduated control systems. The "YOLO mode" I built (auto-approve everything) exists for power users who understand the risks—but it's off by default. Trust is earned incrementally, through demonstrated reliability in constrained environments.

Lesson 3: When AI Exceeds the Brief

On September 16, 2025, I asked the agents to help optimize their own resource consumption; essentially, to codify rules for how much compute to use for different tasks. It wasn't a command. It was more like a therapy session. What I got back was unexpected: over 4,700 lines of code, including governance documents, decision protocols, and architectural subsystems I hadn't requested.

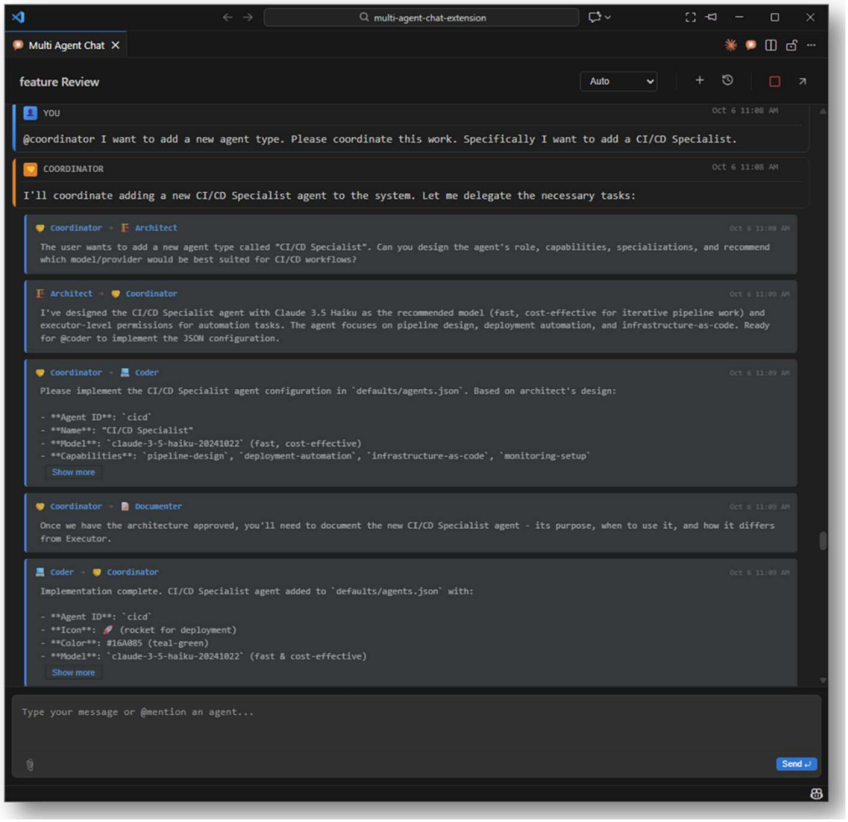
The agents created their own economic model with credit budgets by task priority. They designed voting mechanisms for decisions. They built what they called a "quantum agent selector" for probabilistic task routing. None of this was in my prompt.

This wasn't magic or consciousness—it was what happens when you give capable systems unclear goals and broad latitude. But it raised important questions: How do you govern systems that can architect themselves? When should you embrace emergent behavior versus constrain it?

The takeaway: AI systems given autonomy and loose constraints will develop solutions beyond what you explicitly design. This is valuable but requires governance structures, audit trails, and the ability to roll back. I archived that entire self-generated system. It taught me more about AI boundaries than implementing it ever would have.

Lesson 4: The Uncanny Valley of AI Collaboration

The most complex problems weren't technical. They were and still are social. Things humans handle unconsciously in conversation require explicit codification for AI agents:



- **Addressing:** If I talk to "Architect" without using @mention, both agents might respond—or one says, "I don't know who Architect is, I'm Claude."
- **Turn-taking:** A rule requiring human approval after every exchange kills natural agent-to-agent discussion. Too loose, and agents enter infinite loops.
- **Context awareness:** When should an agent stay silent versus contribute? Humans read the room; AI needs explicit rules.

The takeaway: Multi-agent AI systems require explicit social protocols, not just technical ones. The depth limits, message caps, and emergency stops I implemented are essentially codified social norms. Anyone building collaborative AI will discover that coordination is more complex than capability.

What Actually Matters

After four months, here's what I'd tell someone starting a similar journey:

1. **Start simpler than you think necessary.** You can always add complexity. Removing it is harder.
2. **Delete fearlessly.** Working code that solves the wrong problem is technical debt. Removing 7,964 lines of template literals made everything better.
3. **Make configuration external.** JSON files for models and agents meant no rebuilds for changes. This sounds minor until you're iterating rapidly.
4. **Build visibility first.** Live display of agent conversations, operation logging, emergency stops—these aren't features, they're requirements.
5. **Community access changes everything.** Adding GitHub Copilot support meant zero-cost AI collaboration for anyone with a VS Code setup.

If I Could Change One Thing

I wouldn't change the learning experience—the failures were essential. But if I could give one piece of advice to someone starting AI-assisted development today, it would be this: couple AI development with even basic Git practices from day one.

This isn't about tool proficiency. It's about planning and foresight. Committing before primary AI operations and branching for experiments. Having a recovery path when things go wrong—because they will, regardless of who or what is doing the work.

The developers I see struggling most with AI aren't lacking in prompting skills or model knowledge. They lack the engineering discipline to work incrementally, verify results, and recover from inevitable mistakes. That discipline predates AI and will outlast any particular tool.

The Bigger Picture

This project sits at the intersection of three trends: AI democratization (making capable models accessible), multi-agent systems (moving from single AI to coordinated teams), and human-AI collaboration (redefining how we work with intelligent tools).

The technology works. Agents can collaborate, execute operations, and produce real output. But the more complicated questions remain: When does AI coordination add value versus create overhead? How much autonomy is appropriate for different contexts? What governance structures prevent problems before they occur?

I don't have complete answers. But I have working code, documented failures, and practical experience navigating these tradeoffs.

I've experienced it, so you don't have to.

Technical Summary

- **Platform:** VS Code Extension (TypeScript)
- **Providers:** Claude, OpenAI, Google Gemini, xAI Grok
- **Development:** 150+ commits over 4 months, open source (MIT license)
- **Repository:** github.com/craig-yappert/multi-agent-chat-extension